

Blackjack Insights Engine

A behavioral-analytics audit of `utils/statsEngine.js` — verified by feeding it deliberately patterned mock betting sequences and checking whether the numbers it reports match what those patterns should produce by construction.

Date Aug 20, 2026 **Module under test** `utils/statsEngine.js` **Consumer** `screens/InsightsScreen.js`
Harness Node 24, 14 test suites

14	48	48	100%	1
TEST SUITES	ASSERTIONS	PASSED	PASS RATE	FINDING

01 Purpose & Scope

Why this module gets audited like a ledger, not just spot-checked.

Ante's Blackjack Insights tab tells a player things like *"you bet 43% more after a loss than after a win"* or *"your win rate drops 60 points after two losses."* Those are behavioral claims about a person's own money, generated entirely by `utils/statsEngine.js` — a 348-line pure-calculation module with no UI and no I/O. If a formula in there is subtly wrong, a player doesn't see a broken button; they see a confident, wrong statement about their own gambling behavior.

That risk profile is why this audit doesn't reach for real or randomly-generated session data. Random data can look plausible while silently laundering a bug — a broken streak counter still produces *some* number, and nothing about it looks obviously wrong. Instead, every test below constructs a hand sequence where the "correct" answer is known in advance, by hand, before the engine ever sees it: a martingale betting ladder, a forced win/loss streak, a bet distribution engineered to fall into specific tiers. If the engine's output doesn't match the hand-derived answer, that's a defect, not a judgment call.

In scope: all 14 exported calculation functions consumed by the Blackjack insights view — outcome breakdown, return/ROI math, streaks, conditional (Markov-style) win rates, doubling performance, natural-blackjack frequency, bet-size tiering, volatility scoring, day-of-week and session-length performance, and split-hand flattening.

Out of scope: React Native rendering in `InsightsScreen.js`, AsyncStorage persistence in `storageService.js`, and the live betting UI in `BlackjackScreen.js` (its `calcNet` payout formula is exercised here only as an input generator, reproduced exactly, so mock hands carry realistic `netChange` values).

02 Method

Pattern in, pattern out — or the engine fails.

- 1 **Construct a hand sequence with a known signal.** E.g. for loss-chasing: bet \$10, lose, double to \$20, lose, double to \$40, win, reset to \$10 — a textbook martingale ladder built by hand.
- 2 **Derive the expected statistic independently of the engine.** Average bet after a loss across that ladder is arithmetic anyone can check on paper — \$36.67 across six post-loss bets.
- 3 **Run the real module against it.** `statsEngine.js` was copied byte-for-byte into an isolated Node ESM harness (it has zero React Native dependencies, so this required no mocking) and driven through mock `sessionHistory` objects shaped exactly like `SessionContext` produces — including the newest-first storage order the code reverses internally.
- 4 **Assert, and pair every "pattern" test with a control.** Where it mattered, an unpatterned counterpart was run alongside the engineered one — e.g. a fixed-seed pseudo-random 200-hand shuffle at a ~50% baseline — to confirm the engine reports a *flat* result when there's genuinely nothing to find, not just a positive result when there is.

In the interest of transparency: the first full run scored 45/48. Both failures traced back to flawed assumptions in the mock data itself, not the engine — a strictly-alternating win/loss pattern turned out to be *maximal anti-correlation*, not "no pattern," and an early volatility test didn't swing hard enough to cross the High-risk threshold. Both were corrected and rerun; the log below reflects the final, clean pass.

03 Results Ledger

Every suite: the pattern injected, and what the engine reported back.

01

Streak Detection

MATCH

INJECTED PATTERN

W W W L L W W W L

ENGINE OUTPUT

Longest win 4 · longest loss 2 · current 1L

02

Martingale Bet-Chasing

MATCH

INJECTED PATTERN

Double stake after every loss, reset after a win

ENGINE OUTPUT

Avg bet after loss \$36.67 vs. after win \$10.00 – chasing correctly flagged

03

Flat, Disciplined Bettor

MATCH

INJECTED PATTERN

Constant \$25 stake, mixed outcomes

ENGINE OUTPUT

After-win = after-loss = \$25.00 · consistency 100/100

04

Injected Streak Dependency

MATCH

INJECTED PATTERN

5 losses · 5 wins · 5 losses · 5 wins (clustered, not random)

ENGINE OUTPUT

After 2 losses 25.0% win · after 2 wins 85.7% win – 60.7-pt gap detected

04b

Anti-Correlated & Independent Controls

MATCH

INJECTED PATTERN

Strict W/L alternation (12) · fixed-seed random shuffle (200, ~50% base)

ENGINE OUTPUT

Alternation: 0% / 100% (perfect anti-correlation, correctly caught) · shuffle: 5.5-pt gap only

05

Doubling Performance

MATCH

INJECTED PATTERN

5 doubled hands, all wins · 10 non-doubled, 4W / 6L

ENGINE OUTPUT

Doubled 100% WR / +100% ROI · not doubled 40% WR / -20% ROI · frequency 33.3%

06

Natural Blackjack Frequency

MATCH

INJECTED PATTERN

10 naturals / 100 hands (hot) · 0 naturals / 50 hands (cold)

ENGINE OUTPUT

10.0% vs. 4.8% baseline (hot) · 0.0% (cold) – both read correctly

07

Bet-Size Tiering

FINDING

INJECTED PATTERN

Three clean bet levels – \$10 / \$50 / \$200 – each with a distinct win rate

ENGINE OUTPUT

Tercile boundary ties merged \$10 and \$50 into one 16-hand bucket – see §04

07b

Bet-Size Tiering — Control

MATCH

INJECTED PATTERN

12 distinct, non-tied bet sizes (\$5-\$180)

ENGINE OUTPUT

Cleanly split into 3 non-empty buckets (5 / 4 / 3) – confirms \$07 is tie-specific

08

Volatility Scoring

MATCH

INJECTED PATTERN

Nine \$10 bets + one \$500 spike, vs. ten flat \$10 bets

ENGINE OUTPUT

Spike: **High** risk (ratio 2.69) • flat: **Low** risk (ratio 1.05)

09

Day-of-Week Pattern

MATCH

INJECTED PATTERN

Monday sessions always +\$100 net • Friday sessions always -\$100 net

ENGINE OUTPUT

Best: Monday +\$100.00 • Worst: Friday - \$100.00

10

Session-Length Pattern

MATCH

INJECTED PATTERN

Short (5-hand) sessions all winning • long (30-hand) sessions all losing – a tilt signature

ENGINE OUTPUT

Short +\$20.00/hand • long -\$20.00/hand

11

Split-Hand Flattening

MATCH

INJECTED PATTERN

single → split (2 hands) → single, in that chronological order

ENGINE OUTPUT

4 flattened hands, original order preserved exactly

12

Minimum-Sample Guards

MATCH

INJECTED PATTERN

<6 hands • 1 distinct day • 2 sessions – below each function's stated floor

ENGINE OUTPUT

All three correctly return `null` instead of a misleadingly small-sample stat

13

Payout Math — 3:2 & Doubling

MATCH

INJECTED PATTERN

Doubled win • natural blackjack • plain loss • push, \$100 stake each

ENGINE OUTPUT

Net +\$200 / +\$150 / -\$100 / \$0 • wagered \$500 • ROI +50.0%

14

End-to-End Smoke Test

MATCH

INJECTED PATTERN

120 mixed hands through the full `computeInsights()` aggregator

ENGINE OUTPUT

No exceptions • totals internally consistent (W+L+P = hand count)

04 Finding — Bet-Size Tier Boundaries

The one place the math is internally correct but the bucketing is coarser than a player would expect.

NON-BLOCKING

Tied bet amounts can collapse the small/medium tier split

`calcBetTierWinRates` splits hands into small/medium/large by comparing each bet to two tercile cutoffs, using `≤` on both: `bet ≤ tercile1` is "small," `bet ≤ tercile2` is "medium." When several bets tie exactly on a cutoff value, all of them fall into the lower bucket together.

Test 07 made this concrete with a deliberately clean scenario — 8 hands at \$10, 8 at \$50, 8 at \$200:

```
tercile1 = $50   tercile2 = $200
small  (bet ≤ $50) → n=16 (the $10 and $50 groups merged)
medium (bet ≤ $200) → n=8 (only the $200 group)
large  (bet > $200) → n=0 (empty — nothing exceeds tercile2)
```

The reported "small" win rate becomes a blended 37.5% — the true weighted average of the \$10 group's 25% and the \$50 group's 50% — rather than either number on its own. The arithmetic is correct; the bucketing intent (three comparably-sized groups) is not honored when ties cluster at a boundary.

This isn't a contrived edge case for this app specifically: players commonly re-bet round chip denominations — \$10, \$25, \$50, \$100 — repeatedly within a session, so exact ties at the tercile cutoff are closer to the common case than the exception. The practical effect is a "small bet" tier that quietly absorbs a chunk of what a player would consider their medium bets.

Suggested remediation: compute terciles over the set of *distinct* bet values rather than the raw (duplicate-heavy) array, or switch the upper edge of each bucket to a strict `<` with the final bucket catching the remainder — either change keeps duplicate-heavy, round-number betting from collapsing two intended tiers into one.

05 Conclusion

What this validates, and what it doesn't.

Every behavioral claim the Blackjack Insights tab is capable of making — that a player is chasing losses, riding or fighting a streak, over- or under-doubling relative to basic strategy, running hot or cold on naturals, or trending worse the longer a session runs — was reproduced from first principles in this audit and matched, to the decimal, against the engine's actual output. Just as importantly, the paired control tests (§04b, §07b) confirm the engine stays quiet when a pattern genuinely isn't there, rather than manufacturing a signal from noise.

One finding came out of the process: bet-size tiering (§07) is arithmetically sound but can under-separate tiers when a player's bets tie at exact round numbers, which is the realistic common case rather than a rare one. It's rated non-blocking — nothing it produces is *wrong*, only coarser than intended — but worth fixing before the tiering feature is leaned on further.

This audit covers the calculation engine only. UI rendering, on-device persistence, and the live betting form were exercised as input generators (§02) but not independently verified here.

► **Appendix – full raw assertion log (48/48)**

Scope note. This report covers `utils/statsEngine.js` only, exercised through a standalone Node harness against hand-constructed mock data — no live session data, device storage, or UI code was touched. Prepared for internal engineering review of the Ante blackjack insights feature.